

Desarrollo Web con Python

Flask, Django y la nueva era del Vibe Coding

Emanuel Godoy
Lucciano Martinez
Emmanuel Barcelo
Juan Tucznio
Livio Toribio
Tomás Caballero

Programación 1 | Tecnicatura en Ciencia de Datos | EducacionIT | 2026

Profesor: Juan Pablo Sosa

Índice

1. Flask: cuando menos es más

- 1.1 ¿Qué es Flask?
- 1.2 Rutas
- 1.3 Templates con Jinja2
- 1.4 Request y Response

2. Django: las pilas incluidas

- 2.1 ¿Qué es Django?
- 2.2 Flask vs. Django
- 2.3 El patrón MTV
- 2.4 Model: los datos
- 2.5 View: la lógica
- 2.6 Template: el HTML
- 2.7 Panel de administración

3. Vibe Coding: programar en lenguaje natural

- 3.1 Origen y concepto
- 3.2 Cómo funciona en la práctica
- 3.3 Lo que acelera y lo que no reemplaza
- 3.4 Por qué los fundamentos siguen siendo necesarios

1. Flask: cuando menos es más

1.1 ¿Qué es Flask?

Flask es un microframework web para Python. La palabra *micro* no significa que sea limitado, sino que su núcleo es minimalista: entrega lo estrictamente necesario para levantar un servidor web, y el desarrollador decide qué agregar según las necesidades del proyecto.

La analogía más precisa es la de una caja de herramientas: trae lo esencial y el programador construye con precisión lo que necesita. Esto lo convierte en una opción ideal para APIs, prototipos rápidos y aplicaciones de pequeña a mediana escala.

*Para instalar Flask basta con ejecutar **pip install flask**. En segundos el framework está disponible para usar.*

1.2 Rutas

Una ruta es la conexión directa entre una URL y una función Python. El decorador **@app.route** le indica a Flask qué función ejecutar según la dirección que escribe el usuario en el navegador. Flask también permite rutas dinámicas, donde parte de la URL varía según el usuario, lo que resulta útil para perfiles, productos o cualquier contenido personalizado.

1.3 Templates con Jinja2

Cuando una aplicación web necesita devolver HTML con datos dinámicos en lugar de texto plano, Flask utiliza Jinja2, un motor de templates que permite mezclar HTML estático con variables de Python entre dobles llaves **{{ }}**. Flask procesa el archivo HTML, reemplaza las variables por sus valores reales y entrega la página completa al navegador. El usuario final nunca ve el código Python subyacente.

Los archivos HTML deben ubicarse en una carpeta llamada **/templates** dentro del proyecto, respetando la convención de Flask.

1.4 Request y Response

Toda interacción en la web tiene dos partes: el **request** (lo que el usuario envía al servidor) y el **response** (lo que el servidor devuelve). Flask distingue entre solicitudes **GET** —cuando el usuario visita una página— y **POST** —cuando envía datos a través de un formulario.

Para aplicaciones que devuelven datos estructurados en lugar de HTML, Flask provee la función **jsonify**, que convierte un diccionario Python en formato JSON. JSON es el estándar mundial para el intercambio de datos entre sistemas.

2. Django: las pilas incluidas

2.1 ¿Qué es Django?

Django es un framework web completo para Python. Se lo describe habitualmente como *batteries included* (las pilas incluidas) porque al crear un proyecto Django ya están disponibles la base de datos, el sistema de usuarios, el panel de administración, las rutas y los templates, sin necesidad de instalar nada adicional.

Es uno de los frameworks más utilizados en el mercado laboral, especialmente en empresas que desarrollan productos de mediana y gran escala.

2.2 Flask vs. Django

Criterio	Flask	Django
Punto de partida	Minimalista	Todo incluido
Curva de aprendizaje	Más fácil de arrancar	Más para aprender
Base de datos	Se elige e instala aparte	Incluida con ORM
Panel de admin	No incluido	Automático
Autenticación	La construye el desarrollador	Incluida
Mejor para	APIs y proyectos chicos	Aplicaciones completas
Uso en empresas	Startups y microservicios	Productos de gran escala

Regla práctica: Si se necesita construir algo rápido o una API, se usa Flask. Si se está construyendo un producto completo con usuarios, base de datos y panel de administración, se usa Django.

2.3 El patrón MTV

Django organiza el código siguiendo el patrón **MTV** (Model, Template, View). Cada componente tiene una responsabilidad única y estrictamente definida:

- **Model:** define la estructura de los datos y cómo se almacenan en la base de datos.
- **Template:** es el HTML que ve el usuario en el navegador.
- **View:** es la lógica que conecta los datos con el HTML.

2.4 Model: los datos

En Django no se escribe SQL directamente. Los modelos son clases Python que Django convierte automáticamente en tablas de base de datos. Esto se denomina **ORM** (Object Relational Mapper). Para aplicar los cambios se utilizan dos comandos: **makemigrations** (detecta los cambios) y **migrate** (los aplica). Las migrations funcionan como un historial de versiones de los datos.

2.5 View: la lógica

Las views son funciones que reciben un request, realizan operaciones con los datos y devuelven una response. Son el puente entre los modelos y los templates. Se definen en el archivo **views.py** y se conectan a sus URLs correspondientes en el archivo **urls.py**.

2.6 Template: el HTML

Los templates de Django funcionan de manera similar a los de Flask. Se escribe HTML con variables entre dobles llaves que Django reemplaza por los valores reales al momento de renderizar la página. El usuario final sólo recibe el HTML terminado.

2.7 Panel de administración

Una de las características más valoradas de Django en el entorno laboral es su panel de administración automático. Con una sola línea de código (**admin.site.register(Modelo)**), Django genera una interfaz web completa para crear, editar y eliminar registros de la base de datos. Esto permite tener un backend funcional en minutos, sin construir nada desde cero.

3. Vibe Coding: programar en lenguaje natural

3.1 Origen y concepto

En febrero de 2025, **Andrej Karpathy**, cofundador de OpenAI y ex director de IA de Tesla, popularizó un concepto que rápidamente se volvió viral en la comunidad de desarrollo: el **Vibe Coding**. La idea central es que en lugar de escribir cada línea de código manualmente, el programador describe en lenguaje natural lo que quiere construir, la IA genera el código, y el programador revisa, prueba y pide ajustes.

"En vez de escribir cada línea manualmente, le describís a la IA lo que querés construir, ella genera el código, y vos revisás, probás y pedís ajustes. Sos el director, no el operario."

3.2 Cómo funciona en la práctica

Las herramientas más utilizadas actualmente son **GitHub Copilot**, **Cursor** y **Claude Code**. El flujo de trabajo cambia respecto a la programación tradicional:

- **Programación tradicional:** pensar el algoritmo → escribir línea por línea → buscar errores → corregir → repetir.
- **Vibe Coding:** describir en lenguaje natural → la IA genera el bloque completo → el programador revisa y refina.

Según un informe de GitHub de 2024, más del 55% del código nuevo en plataformas que utilizan Copilot ya está asistido por IA. No es una tendencia futura: es el presente del desarrollo de software.

3.3 Lo que acelera y lo que no reemplaza

La IA hace de manera eficiente:

- Generar código estándar y repetitivo (boilerplate).
- Prototipar ideas rápidamente.
- Explicar errores y documentar sistemas.

El rol irremplazable del programador:

- Definir la arquitectura y qué construir.
- Detectar errores lógicos sutiles.
- Entender el negocio y las reglas de dominio.
- Garantizar la seguridad y escalabilidad del sistema.

3.4 Por qué los fundamentos siguen siendo necesarios

El Vibe Coding no hace irrelevante aprender a programar. Todo lo contrario: eleva la importancia de los fundamentos. Un programador que no entiende el código generado por la IA no puede detectar errores lógicos, no puede mantenerlo ni escalarlo, y no puede justificar por qué el sistema hace lo que hace.

Vibe Coding + fundamentos técnicos = un arquitecto experto utilizando AutoCAD. Multiplica su capacidad de construcción.

Vibe Coding sin saber programar = alguien que hace clic en botones sin entender si el edificio se va a derrumbar.

El futuro del desarrollo pertenece a quienes entienden los fundamentos y apalancan las herramientas.